

Logic in AI : SAT Solvers : DPLL

Algorithm - Part - 7

Introduction

In the realm of Artificial Intelligence, tackling complex computational challenges often boils down to solving fundamental problems. One such cornerstone is the Satisfiability Problem (SAT), a canonical NP-complete problem. This means that if you encounter a difficult problem that falls within the NP complexity class – as many real-world practical problems do – it can theoretically be converted into a SAT problem. By solving the SAT instance, you effectively solve the original problem. This lecture delves into the intricacies of SAT problems, exploring how they are formulated and, more importantly, how they can be efficiently solved using advanced algorithms.

A SAT problem involves a set of logical variables and a formula expressed in Conjunctive Normal Form (CNF), which is a conjunction of clauses, where each clause is a disjunction of literals. The objective is to find an assignment of truth values (true or false) to these variables such that all clauses in the CNF formula are simultaneously satisfied. This process is often referred to as "model finding," as a complete assignment of variables that satisfies the formula constitutes a "model" in logical terms. While a brute-force approach of checking all possible truth assignments is conceptually straightforward, its exponential time complexity (2^n for 'n' variables) renders it impractical for all but the smallest problems. This necessitates more sophisticated techniques, leading us to the Davis-Putnam-Logemann-Loveland (DPLL) algorithm, a significant advancement in SAT solving.

Core Concepts

Term	Definition	Significance
Satisfiability Problem (SAT)	A decision problem asking if there exists an assignment of truth values to variables that makes a given Boolean formula true.	It is a canonical NP-complete problem, meaning many hard problems can be reduced to it.
Conjunctive Normal Form (CNF)	A logical formula that is a conjunction of clauses, where each clause is a	Standardized input format for most SAT solvers,

Term	Definition	Significance
	disjunction of literals (variables or their negations).	simplifying algorithm design.
Model Finding	The process of finding a complete assignment of truth values to variables that satisfies a given logical formula.	The core objective of SAT solving; a satisfying assignment is a "model."
Brute-Force Search	Checking every possible truth assignment for 'n' variables (2^n possibilities) to find a satisfying one.	Conceptually simple but computationally infeasible due to exponential time complexity.
Davis-Putnam-Logemann-Loveland (DPLL)	A complete, backtracking-based search algorithm for deciding the satisfiability of propositional logic formulas in CNF.	A foundational and highly influential algorithm for SAT solving, developed in the 1960s.
Unit Literal	A clause containing only a single literal (e.g., (A) or ($\neg B$)).	Allows immediate assignment of the variable to satisfy the clause, reducing search space.
Unit Propagation	The process of repeatedly applying the unit literal rule to simplify the formula and make further assignments.	A powerful constraint propagation technique that significantly prunes the search tree.
Pure Literal	A literal that appears with only one polarity (either always positive or always negative) across all clauses in the formula.	Allows immediate assignment of the variable to satisfy all clauses it appears in without hurting others.
Pure Literal Elimination	The process of assigning a truth value to a pure literal	Another effective constraint propagation technique that

Term	Definition	Significance
	to satisfy all clauses it appears in.	avoids unnecessary branching.
MOM's Heuristic	A branching heuristic that selects the variable appearing most frequently in clauses of minimum length.	A strategy to guide the search by prioritizing variables likely to create new unit clauses quickly.

Detailed Analysis

1. The SAT Problem and Brute-Force Limitations

The Satisfiability Problem (SAT) is central to many computational tasks. It involves a Boolean formula, typically presented in Conjunctive Normal Form (CNF), which is a conjunction of clauses, where each clause is a disjunction of literals. For example, $(A \vee B \vee C) \wedge (\neg B \vee \neg C)$. The goal is to find a truth assignment for the variables (A, B, C) that makes the entire formula true.

A naive approach would be to enumerate all possible truth assignments. For n variables, there are 2^n possible assignments. Each assignment would then be checked for satisfiability against the formula. This "brute-force" method, while guaranteed to find a solution if one exists, is computationally prohibitive for even moderately sized problems due to its exponential time complexity. This limitation highlights the need for more intelligent search strategies. The SAT problem can be viewed as a Constraint Satisfaction Problem (CSP), where variables are Boolean and clauses act as constraints. This connection suggests that search techniques developed for CSPs, such as backtracking, could be adapted for SAT.

2. Introducing the DPLL Algorithm

The Davis-Putnam algorithm, later refined into the Davis-Putnam-Logemann-Loveland (DPLL) algorithm in the 1960s, was a pioneering effort to solve SAT problems more efficiently than brute-force. DPLL is essentially a backtracking search algorithm. It operates by taking a partial assignment of variables and recursively trying to extend it. If a partial assignment makes the formula false, it backtracks. If it makes the formula true, it returns true. Otherwise, it chooses an unassigned variable and branches, trying both `true` and `false` assignments for that variable. This branching factor is always 2.

The lecture briefly touches upon a philosophical question: are algorithms "invented" or "discovered"? While a revolver is clearly an invention, and a geographical location a discovery, the nature of algorithms like Dijkstra's or Depth-First Search is more ambiguous. Do they pre-exist, waiting to be found, or are they constructed by human ingenuity? This thought-provoking diversion underscores the creative and analytical aspects of computer science.

3. Basic DPLL Workflow and Limitations

A very basic DPLL algorithm would proceed as follows:

1. If the formula is satisfied by the current assignment, return true.
2. If the current assignment leads to a contradiction (e.g., an empty clause), return false and backtrack.
3. Otherwise, pick an unassigned variable.
4. Recursively try assigning it `true`. If that leads to a solution, return true.
5. If not, backtrack and recursively try assigning it `false`. If that leads to a solution, return true.
6. If neither assignment works, return false.

This basic backtracking search, similar to depth-first search in CSPs, explores the space of partial assignments. However, it is still too simplistic and can take a long time for many problems, as it doesn't incorporate any intelligence beyond simple branching.

4. Improving DPLL: Constraint Propagation

To make DPLL more efficient, several constraint propagation techniques are introduced, similar to those used in CSPs:

- * **Clause Removal:** If a clause contains a literal that has already been assigned `true`, that clause is satisfied and can be removed from the formula.
- * **Literal Shortening:** If a variable in a clause is assigned a value that negates its literal in that clause (e.g., variable `A` is set to `false`, and the clause contains `A`), then that literal can be removed from the clause.
- * **Empty Clause Detection:** If, after literal shortening, a clause becomes empty, it means that the current partial assignment leads to a contradiction. The algorithm must then backtrack.
- * **Empty Formula Detection:** If all clauses are removed (meaning the formula is satisfied), the algorithm returns `true`.

These improvements allow the algorithm to simplify the formula dynamically during search, potentially detecting failures earlier or finding solutions faster. For instance, if `A` is set to `false` in a formula, a clause `(A OR B OR C)` becomes `(B OR C)`. If another clause `(NOT A OR D)` becomes satisfied and is removed.

5. Advanced DPLL Techniques: Unit Propagation

A significant improvement comes from identifying "unit literals." A unit literal is a clause that contains only one literal (e.g., (C) or $(\text{NOT } B)$). If such a clause exists, the variable in that literal must be assigned a specific value to satisfy the clause. For example, if (C) is a clause, C must be `true`. If $(\text{NOT } B)$ is a clause, B must be `false`.

The process of repeatedly identifying unit literals and making the necessary assignments, then propagating their effects throughout the formula (shortening clauses, removing satisfied clauses), is called **Unit Propagation**. This is analogous to "forward checking" in CSPs, where assignments immediately prune the domain of other variables. Unit propagation is powerful because it allows the algorithm to make deterministic assignments without branching, significantly reducing the search space.

Consider the formula: $(A \text{ OR } B) \text{ AND } (\text{NOT } B \text{ OR } C) \text{ AND } (\text{NOT } C)$.

1. The clause $(\text{NOT } C)$ is a unit literal. C must be `false`.
2. Propagate $C = \text{false}$:
 - * $(\text{NOT } B \text{ OR } C)$ becomes $(\text{NOT } B \text{ OR } \text{false})$, which simplifies to $(\text{NOT } B)$.
3. Now $(\text{NOT } B)$ is a unit literal. B must be `false`.
4. Propagate $B = \text{false}$:
 - * $(A \text{ OR } B)$ becomes $(A \text{ OR } \text{false})$, which simplifies to (A) .
5. Now (A) is a unit literal. A must be `true`.
6. All clauses are satisfied. The solution is $A=\text{true}, B=\text{false}, C=\text{false}$.

In this example, no branching was required; the solution was found purely through inference.

6. Advanced DPLL Techniques: Pure Literal Elimination

Another powerful technique is "Pure Literal Elimination." A literal is "pure" if it appears with only one polarity (either always positive or always negative) across all clauses in the current formula. For example, if D appears as D in some clauses but never as $\text{NOT } D$, then D is a pure literal.

If a literal L is pure, assigning L to `true` (if it's positive) or `false` (if it's negative) will satisfy all clauses containing L without making any other clause harder to satisfy. Since the goal is to find any satisfying assignment, we can immediately set the pure literal to its satisfying value without needing to branch. This is a "good decision" that doesn't require backtracking.

Example: $(A \text{ OR } B) \text{ AND } (\text{NOT } A \text{ OR } C) \text{ AND } (D \text{ OR } B)$

- * Notice D appears only as D (positive form). D is a pure literal. We can set $D = \text{true}$.

- * The clause $(D \vee B)$ is now satisfied and can be removed.
- * The remaining formula is $(A \vee B) \wedge (\neg A \vee C)$.
- * Now, B appears only as B (positive form). B is a pure literal. We can set $B = \text{true}$.
- * The clause $(A \vee B)$ is now satisfied and can be removed.
- * The remaining formula is $(\neg A \vee C)$.
- * Now, C appears only as C (positive form). C is a pure literal. We can set $C = \text{true}$.
- * The clause $(\neg A \vee C)$ is now satisfied.
- * The solution is $D=\text{true}, B=\text{true}, C=\text{true}$. A can be anything.

The final DPLL algorithm incorporates these improvements: if a unit literal or a pure literal exists, it is immediately assigned, and its effects are propagated. Only when neither of these conditions applies does the algorithm resort to branching.

7. Branching Heuristics for DPLL

When no unit literals or pure literals are present, the DPLL algorithm must choose a variable to branch on. The choice of which variable to pick can significantly impact performance. This is where branching heuristics come into play.

In CSPs, a common heuristic is to pick the "most constrained variable" (e.g., the one with the fewest remaining values). However, in SAT, all variables initially have two possible values (true/false), so this specific heuristic doesn't directly apply. Instead, the goal is to make choices that are likely to lead to quick simplifications or early detection of contradictions.

A popular heuristic for DPLL is the **MOM's (Most Occurrences in Minimum Length Clauses) heuristic**. The intuition behind MOM's is to prioritize variables that are part of "small" clauses (e.g., length 2 or 3). Small clauses are more restrictive; satisfying or falsifying a literal in a small clause has a higher chance of creating a unit clause or an empty clause. By picking a variable that appears frequently in these critical short clauses, the algorithm aims to quickly reduce the formula and potentially trigger unit propagation, thereby accelerating the search. For example, if there are many clauses of length 2, the heuristic would pick the variable that appears most often in those length-2 clauses.

Key Takeaways

- [] SAT problems are canonical NP-complete problems, making their efficient solution crucial for many AI applications.
- [] Brute-force search for SAT is impractical due to its exponential time complexity (2^n).

- [] The DPLL algorithm is a backtracking search method for SAT, significantly more efficient than brute-force.
- [] DPLL incorporates constraint propagation techniques like clause removal and literal shortening to simplify the formula.
- [] Unit Propagation is a key optimization where single-literal clauses (unit literals) force immediate variable assignments, avoiding branching.
- [] Pure Literal Elimination allows immediate assignment of variables that appear with only one polarity, further reducing the search space.
- [] When no unit or pure literals exist, branching heuristics are used to select the next variable to assign.
- [] The MOM's heuristic (Most Occurrences in Minimum Length Clauses) is a common branching strategy that prioritizes variables in short, restrictive clauses to accelerate propagation.

Conclusion

The DPLL algorithm, through its evolution from a basic backtracking search to an intelligent system incorporating unit propagation, pure literal elimination, and sophisticated branching heuristics, stands as a testament to the power of informed search in AI. By leveraging logical inferences and strategic choices, DPLL transforms an intractable exponential problem into one that can be solved for many real-world instances. This algorithm forms the backbone of modern SAT solvers, which are used in diverse fields such as hardware verification, automated planning, scheduling, and even in solving complex mathematical problems. Understanding DPLL not only provides insight into a fundamental AI problem-solving technique but also highlights the continuous interplay between theoretical computer science and practical algorithmic design.